Cuadrados Mágicos

Un cuadrado mágico es una matriz cuadrada de números enteros en la que la suma de sus columnas, filas y diagonales principales, dan como resultado el mismo valor, el cual es conocido como número mágico o constante mágica.

Por ejemplo, el siguiente es un cuadrado mágico de orden 3, en donde la constante mágica es 15:

8 3 4

1 5 9

6 7 2

Entradas

La entrada comienza con un número entero positivo n, que representa el orden del cuadrado a evaluar.

A continuación, vendrán n x n números enteros, que componen la matriz. Los primeros n números corresponden a la primera fila, los siguientes n a la segunda, y así sucesivamente.

Salidas

Si la matriz ingresada es un cuadrado mágico, la salida debe ser un único número entero correspondiente a la constante mágica del cuadrado. En caso contrario, la salida debe ser la palabra NO (todo en mayúsculas).

Observaciones

Puede suponer que el valor de n será siempre un entero positivo. Asimismo, los valores ingresados para la matriz serán todos enteros. No es necesario hacer validaciones.

Por ejemplo:

Entrada	Resultado
3 8 3 4 1 5 9 6 7 2	15
4 3 3 7 4 2 5 1 8 7 6 2 1 1 2 3 4	NO

Mediana Doble

Dada una muestra de valores positivos y **ordenados**, se define la mediana como el valor que ocupa la posición central de los datos dados:

1 2 6 17 18 — **22** — 35 46 109 143 200

En el caso de tener un número impar de valores, la mediana está clara: será aquel valor que tenga el mismo número de valores más pequeños y más grandes que él en la muestra. En el caso de tener un número par de valores, habría dos candidatos a ser mediana. En vez de decidirnos por uno, en este caso la mediana viene dada por la media aritmética de esos dos valores que ocupan las posiciones centrales.

Escriba un programa que calcule y muestre como salida **el doble de la mediana** de una colección de números enteros positivos que serán ingresados como entrada.

Entradas

Comienza con un número que indica la cantidad de valores que tiene la muestra. A continuación se dan los valores de la muestra, todos números enteros positivos desordenados, de los que habrá que calcular la mediana multiplicada por dos.

Salidas

Para la muestra de entrada se mostrará **el doble** de la mediana de los valores dados en **formato entero**

Observaciones

Puede suponer que las entradas cumplen siempre con el formato indicado y que los números de la colección **no** están ordenados.

Por ejemplo:

Entrada	Resultado
11 1 2 6 17 18 22 35 46 109 153 200	44

Filtro

Se cuenta con una lista original que contiene cualquier tipo de elementos, y una lista de elementos a eliminar. Debemos crear una nueva lista que contenga únicamente los elementos de la lista original que no están en la lista de elementos a eliminar, asegurándose además de que no haya elementos repetidos, manteniendo únicamente la primera ocurrencia de cada valor.

Por ejemplo:

Si la lista original y la lista con valores a eliminar tienen la forma

original = [1, 2, 3, 1, 2, 3, 1, 2, 3]

eliminar = [1, 3]

el retorno de la función deberá ser la lista [2], la cual contiene los elementos de la lista original (sin repetir) que no se encuentran en la lista eliminar.

Escriba la función filtrar(original, eliminar), que recibe como parámetro las dos listas a procesar. La función debe retornar una nueva lista que contiene los elementos de la lista original que no se encuentran en la lista eliminar. Además, en caso de elementos repetidos en la lista original, se deben mantener únicamente la primera ocurrencia.

Entradas

Dos listas de cualquier tipo de elemento.

Salidas

Una lista, construida según el procedimiento descrito.

Observaciones

Recuerde que solo debe entregar la función solicitada y que no debe incluir ni input() ni print() en su código.

Por ejemplo:

Prueba	Resultado
`print(filtrar([1, 2, 3, 1, 2, 3, 1, 2, 3], [1, 3]))`	`[2]`
`print(filtrar(['peras', 1, 2, 3.1415, 'manzanas', 'manzanas', 'peras', 3.1415, 2, 1, 2, 1, 2, 1, 'peras', 'manzanas'], ['peras', 1, 2]))`	`[3.1415, 'manzanas']`
`print(filtrar([1, 2, 1, 2, 2], [1, 2]))`	`[]`

Contar Letras

Se quiere contar la cantidad de veces que aparece cada letra de un texto.

Escriba la función contar_letras(texto), que recibe como parámetro un string con el texto a procesar. La función debe retornar una lista donde cada elemento corresponde a una lista de dos elementos con una letra y la cantidad de veces que aparece esa letra en el texto. La lista aparece ordenada alfabéticamente, de acuerdo a la tabla ASCII. Otros caracteres que aparezcan en el texto y que no son letras **no deben ser tomados en cuenta**. Si el texto no tiene letras que contar, la función debe retornar una lista vacía.

Entradas

Un *string* con el texto a procesar, que está compuesto por letras y signos de puntuación.

Salidas

Una lista ordenada alfabéticamente, donde cada elemento es una lista de dos elementos, con una letra y la cantidad de veces que aparece esa letra en el texto pasado como parámetro. Guíese por los ejemplos.

Observaciones

Las mayúsculas y las minúsculas se consideran letras distintas.

Por ejemplo:

Prueba	Resultado
`print(contar_letras("En un lugar de La Mancha, de cuyo nombre no quiero acordarme"))`	`[['E', 1], ['L', 1], ['M', 1], ['a', 6], ['b', 1],`
`print(contar_letras(""))`	`[]`
`print(contar_letras("hola"))`	`[['a', 1], ['h', 1], ['l', 1], ['o', 1]]`
`print(contar_letras(";,  ;.%"))`	`[]`

Por ejemplo:

```
, ['c', 3], ['d', 3], ['e', 5], ['g', 1], ['h', 1], ['i', 1], ['l', 1], ['m', 2], ['n', 5], ['o', 5], ['q', 1], ['r', 5], ['u', 4], ['y', 1]]
```

Los productos más caros

Dado un listado de productos y sus precios, queremos obtener los nombres de los tres productos más caros, ordenados de mayor a menor de acuerdo a su precio. Escriba un programa que lea como entrada la cantidad de productos, seguida de los nombres y precios de esos productos. Debe mostrar los nombres de los tres productos más caros, de mayor a menor según el precio.

Entradas

La primera entrada es un número entero positivo **n** que indica la cantidad de productos del listado. Puede suponer que siempre habrá al menos 3 productos. Las siguientes **2n** entradas corresponden a los productos y precios. Puede suponer que los nombres y los precios de los productos son únicos (no se repiten).

Salidas

Los nombres de los tres productos más caros, uno por línea, ordenados de mayor a menor según su precio.

Por ejemplo:

Entrada	Resultado
3 Salsa 500 Manjar 1200 Lechuga 300	Manjar Salsa Lechuga

Tiempos de Espera

En la sala de urgencias de un hospital, a cada paciente que ingresa se le calcula el tiempo que deberá esperar para ser atendido. Sin embargo, un equipo de analistas ha notado que el tiempo real que espera un paciente está dado por la siguiente expresión: ter_paciente = ta_paciente + prom_ter_pacientes + n_pacientes_anteriores donde:

- ter_paciente: tiempo de espera real de un paciente
- ta_paciente: tiempo asignado al ingresar
- prom_ter_pacientes: promedio de todos los tiempos asignados de pacientes en espera
- n_pacientes_anteriores: cantidad de pacientes que llegaron antes

Conociendo los tiempos de espera asignados al ingresar a los pacientes, obtenga los tiempos reales de espera para cada uno.

Entradas

La primera entrada, n, corresponde a la cantidad de tiempos de espera que se ingresarán. Suponga que n es un número entero positivo y que es al menos 3. Las siguientes n entradas, corresponden a los tiempos de espera asignados a cada paciente al ingresar. También son números enteros y positivos.

Salidas

Los tiempos de espera real de los pacientes, uno por línea. **Importante:** Los valores deben ser enteros.

Observaciones

Para la salida use el valor truncado del tiempo calculado, utilizando int(). Sin embargo, para los calculos intermedios no utilice aproximaciones ni truncamientos.

Por ejemplo:

Entrada	Resultado
5 10 20 10 35 40	33 44 35 61 67

Fibonacci

La secuencia de Fibonacci es una progresión ordenada e infinita de números que se obtiene con el siguiente algoritmo:

- Se establece dos números base: 0 y 1.
- Cada número siguiente se obtienen como la suma de los dos anteriores.

Así, la secuencia de Fibonacci toma la siguiente forma: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34...

Escriba un programa que solicite un número entero ***n*** que representa la cantidad de valores Fibonacci que desea generar y que imprima una lista con la secuencia generada. Por ejemplo:

- Si n es 1, entonces su programa imprime la lista [0].
- Si n es 2, entonces su programa imprime [0, 1].
- Si n es 10, su programa debe imprimir [0, 1, 1, 2, 3, 5, 8, 13, 21, 34].

Si n es menor que 1, su programa debe responder imprimiendo "Error" (sin comillas en la salida).

Entradas

Un número entero que representa la cantidad de valores Fibonacci que desea generar.

Salidas

Una lista de números enteros con la secuencia generada.

Por ejemplo:

Entrada	Resultado
10	[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

Ruta Turística

Un grupo de tursitas quieren visitar varios lugares de la ciudad. Para esto, recibieron información del tiempo de traslado que toma un paseo por diferentes lugares de interés. Los lugares de interés son cinco: Museo, Plaza, Teatro, Estadio y Restaurant. Se cuenta con una lista de listas con las distancias entre un lugar y otro, como la que se muestra a continuación:

```
d = [
  [0, 4, 7, 3, 10], # Museo
  [4, 0, 2, 8, 6], # Plaza
  [7, 2, 0, 5, 3], # Teatro
  [3, 8, 5, 0, 9], # Estadio
  [10, 6, 3, 9, 0] # Restaurant
]
```

Cada sublista toma como referencia a un lugar y cada número de la sublista es la distancia de ese lugar de referencia hacia otro lugar. Los lugares están siempre en el mismo orden, por lo que, y a modo de ejemplo, la primera sublista indica la distancia del Museo al mismo Museo (distancia 0), luego del Museo a la Plaza (distancia 4), luego del Museo al Teatro (distancia 7) y así sucesivamente. Esto quiere decir, que esta lista de listas también puede ser vista como una matriz donde cada fila y columna representa un lugar y el valor de esa coordenada es la distancia entre ambos lugares. Es por esto que en la diagonal se observan sólo valores ceros (0), ya que la distancia de un lugar con sigo mismo es cero.

Escriba la función distancia_recorrida(d, ruta) que recibe como parámetro la listas de listas con las distancias entre los lugares, y una ruta, que corresponde a una lista de los lugares a visitar. La función debe retornar la distancia total recorrida al pasar por todos esos lugares en el orden indicado y volviendo siempre al lugar de inicio.

Entradas

Los parámetros de la función son:

- d: La lista de listas que contiene las distancias entre los lugares turísticos que se podrían visitar.
- ruta: Una lista de strings con los nombres de los lugares que se quieren visitar. El orden de los elementos en la lista determina el orden en que se visitarán.

Salidas

Un único número que corresponde a la distancia total recorrida, luego de pasar por todos los lugares indicados en la ruta y volviendo siempre al lugar de inicio.

Por ejemplo:

Prueba	Resultado
`print(distancia_recorrida([[0,4,7,3,10],[4,0,2,8,6],[7,2,0,5,3],[3,8,5,0,9],[10,6,3,9,0]], ["Museo", "Plaza", "Teatro", "Estadio", "Restaurant"]))`	30

Paseando Perros

Un paseador de perros necesita organizar a sus cuadrúpedos clientes en grupos para los paseos diarios. A partir de una lista que contiene los nombres de los regalones, y un número entero que indica la cantidad que puede pasear a la vez, debe definir los grupos, asignándolos en el mismo orden en que vienen en la lista original.

Escriba la función organizar(perros, m), que recibe como parámetros una lista de strings con los nombres de los perros y un número entero positivo m que indica el tamaño de los grupos.

La función asignar a los perros, en el mismo orden en que aparecen en la lista del parámetro, en grupos de tamaño m, retornando una lista con una sublista para cada grupo. En las sublistas deben aparecer los nombres de los perros en el grupo.

Si el último grupo tiene menos perros que m, se debe incluir igualmente.

Entradas

La función tiene dos parámetros:

- Una lista de strings con los nombres de los perros.
- El tamaño de los grupos a construir.

Salidas

Una lista de listas, donde cada sublistas contiene los nombres de los perros en un grupo. Los perros se deben asignar en el mismo orden en que vienen en la lista original.

Por ejemplo:

Prueba	Resultado
`print(organizar(["Martín","Peluson","Sanson","Federina","Contento"], 2))`	`[['Martín', 'Peluson'], ['Sanson', 'Federina'], ['Contento']]`

Grupos de Estudio

Queremos ayudar a un conjunto de estudiantes a organizarse para estudiar en parejas. El criterio para construir las parejas asigna a la persona con la nota más alta en el certamen con la persona con la nota más baja. Luego, hará lo mismo con la siguiente nota más alta y la siguiente más baja, y así sucesivamente. Lo que se pretende es que quienes demuestran un mejor desempeño puedan ayudar a quienes todavía están luchando por entender la materia.

Escriba la función hacer_grupos(estudiantes), que recibe como parámetro una lista de listas. Cada sublista corresponde a un o una estudiante, y contiene dos elementos: la nota en el certamen y el nombre. La lista no sigue ningún orden particular. La función debe retornar una lista de listas, con las parejas que fueron asignadas para estudiar, según el criterio descrito. Además de los nombres, cada sublista debe incluir el promedio de las notas obtenidas en el certamen por cada integrante, redondeado al entero más cercano.

Si al finalizar la asignación queda algún o alguna estudiante sin pareja, debe agregarse a la lista, junto con la nota de su certamen.

Entradas

Una lista de listas con el formato:

[[nota1,nombre1],[nota2,nombre2], ...]

Salidas

Una lista de listas con el formato:

[[estudiante1,estudiante2,promedio],[estudiante3,estudiante4], ...]

Si alguien quedó sin pareja al final, se agregará a la lista anterior como la sublista: [nombre, nota].

Observaciones

Estudie cuidadosamente el ejemplo, para comprender el formato de la lista que debe retornar la función, incluído el redondeo.

Por ejemplo:

Prueba
`print(hacer_grupos([[50, 'Ana'], [20, 'Pedro'], [30, 'Juan'], [70, 'Diego'], [100, 'Javi']]))`

Resultado
`[['Pedro', 'Javi', 60.0], ['Juan', 'Diego', 50.0], ['Ana', 50]]`